



Neighbor-aware shared container instance warming framework for serverless edge computing[☆]

Yumi Kim¹, Bokyeong Kim, Taewon Song^{*}, Haneul Ko^{ID}^{*}

Department of Electronic Engineering, Kyung Hee University, Yongin-si, Gyeonggi-do, 17104, Republic of Korea

ARTICLE INFO

Keywords:

Serverless computing
Cloud
Internet of things (IoT)
Container management
Cold start

ABSTRACT

In serverless edge computing, cold starts can hinder timely task execution. In this paper, we propose a neighbor-aware shared container instance warming (NA-SCIW) framework. NA-SCIW employs two key concepts: (1) shared container instances; and (2) a cooperative keep-warming mechanism. By enabling multiple IoT devices to share a container instance, the framework ensures that as long as at least one request is made, the instance remains warm, preventing cold starts. However, if no requests are generated for a certain period, the shared instance transitions to a cold state. To mitigate this, NA-SCIW introduces a cooperative keep-warming mechanism, where each IoT device strategically decides whether to send a virtual request based on the actions of nearby devices. We formulate the problem as a constrained stochastic game model, aiming to minimize cold start occurrences while keeping the average additional traffic overhead and energy outage probability of IoT devices within acceptable limits. To solve this model, we develop an algorithm based on best response dynamics. Evaluation results demonstrate that NA-SCIW effectively reduces cold start probability while ensuring low traffic overhead and minimal energy depletion for IoT devices.

1. Introduction

Serverless computing is increasingly applied to edge environments, combining the benefits of on-demand resource allocation with low-latency edge processing. However, a major challenge in serverless edge computing is the cold start problem, where container instances undergo initialization and image loading before execution. Cold starts introduce significant delays (e.g., 2–5 s [1]), making it difficult to meet real-time processing requirements.

A common approach to mitigate cold starts is keeping container instances warm, allowing them to be reused without reinitialization [2, 3]. In this approach, the container instance remains idle for a short period after completing a task, enabling rapid execution if a new request arrives. However, resource constraints in edge clouds make it impractical to keep all instances warm indefinitely [4]. Thus, when an instance remains unused for a predefined duration, it transitions to a cold state, leading to increased task execution delays.

In this paper, we propose a neighbor-aware shared container instance warming (NA-SCIW) framework. In conventional serverless architectures, each IoT device manages its container instances separately. As a result, devices with low task generation rates often experience cold starts, while allocated resources remain underutilized. NA-SCIW addresses this inefficiency by allowing IoT devices to share container instances for specific tasks, improving resource utilization. With this approach, as long as at least one request is made within a shared group, the instance remains warm, effectively preventing IoT devices from experiencing cold starts. Despite the benefits of shared instances, a challenge arises when no IoT devices generate requests for an extended period, causing the instance to transition to a cold state. To prevent this, NA-SCIW introduces a cooperative keep-warming mechanism, where IoT devices strategically decide whether to send virtual requests based on the decisions of nearby devices. This mechanism optimizes resource usage by ensuring that unnecessary virtual requests are minimized while maintaining warm instances when needed. To formally model this decision-making process, we develop a constrained stochastic game

[☆] This research was supported by National Research Foundation (NRF) of Korea Grant funded by the Korean Government (MSIP) (No. RS-2024-00340698 and RS-2024-00451909).

^{*} Corresponding authors.

E-mail addresses: yumikim1201@khu.ac.kr (Y. Kim), bokyeong0405@khu.ac.kr (B. Kim), twsong@sch.ac.kr (T. Song), heko@khu.ac.kr (H. Ko).

¹ Y. Kim and B. Kim contributed equally to this work.

² Each player in the constrained stochastic game model performs a series of particular actions to minimize (or maximize) the cost (or reward) under some constraints by considering the actions of other players. Since the player's behavior in the game model is the same as the IoT device's behavior in NA-SCIW, the stochastic game model is used in this paper.

<https://doi.org/10.1016/j.future.2025.107986>

Received 18 March 2025; Received in revised form 17 June 2025; Accepted 18 June 2025

Available online 7 July 2025

0167-739X/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Comparison of related works based on the reviewed literature.

Literature	Objective	Consideration aspect		Optimization method
		Architecture design	Optimization	
[5]	Designing a serverless edge analytics platform integrating cloud-edge environments for automated function orchestration and optimized data processing	O	X	–
[6]	Developing a container-based serverless architecture leveraging Docker on AWS Lambda for efficient event-driven processing	O	X	–
[7]	Designing an edge-cloud serverless architecture with dynamic task placement to optimize execution latency and cost	O	X	–
[8]	Designing a serverless architecture for distributed edge computing, integrating ETSI MEC for dynamic resource management and load balancing	O	X	–
[9]	Designing a serverless architecture for edge computing, analyzing IoT-cloud integration challenges and research opportunities	O	X	–
[10]	Designing a scalable serverless data pipeline architecture for IoT analytics in fog and cloud environments	O	X	–
[11]	Designing a serverless architecture for hardware acceleration, introducing HARDLESS for efficient execution of compute-intensive workloads	O	X	–
[12]	Designing a serverless architecture for decentralized IoT dataflows, leveraging Zenoh and WebAssembly for scalable nanoservice execution	O	X	–
[13]	Designing a federated serverless architecture for scientific computing, enabling scalable function execution across supercomputers, clouds, and clusters	O	X	–
[14]	Optimizing container scheduling in serverless edge computing by balancing data movement and execution cost	X	O	Proposing heuristic algorithm
[15]	Optimizing serverless execution by reducing cold starts	X	O	Proposing heuristic algorithm
[16]	Optimizing serverless caching by dynamically adjusting function retention to minimize cold starts	X	O	Formulating integer linear programming
[17]	Optimizing task scheduling in mobile edge computing by minimizing execution cost and energy consumption	X	O	Formulating mixed integer linear programming
[18]	Optimizing workload scheduling in hybrid serverless-serverful computing	X	O	Using deep reinforcement learning
[19]	Optimizing serverless IoT task scheduling by reducing latency and energy use	X	O	Using deep reinforcement learning
[20]	Optimizing serverless resource management by dynamically adjusting memory and keep-alive policies	X	O	Deriving keep-warming policy
[1]	Optimizing serverless performance by reducing cold-start overheads	X	O	Deriving keep-warming policy
NA-SCIW	Designing NA-SCIW, a shared container warming architecture for serverless edge computing, optimizing resource use and cold start mitigation through cooperative keep-warming	O	O	Formulating CMDP

O and X represent that the literature considers and does not consider the corresponding resource, respectively.

model, which aims to minimize cold start occurrences while keeping additional traffic overhead and energy depletion within acceptable limits.² We solve this model using an algorithm based on best response dynamics, ensuring adaptive and efficient decision-making among IoT devices. Evaluation results show that NA-SCIW effectively reduces cold start occurrences while maintaining low additional traffic overhead and minimal energy depletion. Furthermore, the framework adapts dynamically to the IoT environment, allowing devices to optimize their decisions on sending virtual requests based on contextual factors (see Table 1).

This paper makes several key contributions:

- Introduction of NA-SCIW: We propose NA-SCIW, which integrates (1) shared container instances and (2) a cooperative keep-warming mechanism. This approach enhances the probability of warm starts while optimizing edge cloud resource utilization.
- Efficient policy optimization: We develop an efficient algorithm that quickly determines optimal warming policies for IoT devices

with minimal computational overhead, achieving convergence in only a few iterations.

- Comprehensive evaluation: We conduct extensive performance evaluations across diverse scenarios, providing a foundation for the development of serverless edge computing systems tailored for energy-harvesting IoT devices.

Sections 2 and 3 in this paper provide a summary of related works and an explanation of NA-SCIW, respectively. The formulation of the constrained stochastic game model is presented in Section 4. The evaluation results are detailed in Section 5. Section 6 covers the conclusions and future research directions.

2. Related work

Numerous works have been undertaken to improve the performance of the offloading service in serverless computing environments [1,5–23]. These works can be broadly classified into three categories: (1) the design of architecture [5–13]; (2) performance optimization [1,14–

20]; and (3) performance analysis [21–23]. Given that the NA-SCIW framework introduces a serverless computing architecture featuring the shared container instance and an optimization technique for maintaining the instance in a warm status through a distributed approach, this paper falls into both the first category, which pertains to architecture design, and the second category, which involves optimization techniques. Table 1 summarizes the representative related works.

In order to provide a variety of microservices, various architecture designs have been attempted that introduce the concept of serverless into the edge computing environment. Nastic et al. [5] presented a reference architecture designed to simplify the development and management of analytics functions, thus relieving users from the intricacies of the underlying edge infrastructure. Pérez et al. [6] introduced a framework and a methodology that support highly-parallel serverless applications, particularly suited for high-throughput computing tasks. Das et al. [7] presented a container initiation system that determines the best location for container instances by considering both latency and operational costs. Cicconetti et al. [8] outlined a serverless computing implementation in actual systems by leveraging existing edge computing standards. Aslanpour et al. [9] studied whether and how serverless computing can adapt to edge computing and IoT applications. Poojara et al. [10] proposed several serverless data pipeline approaches and applied them to various edge computing applications. Werner et al. [11] introduced an initial conceptualization and a practical realization of a serverless computing architecture capable of accommodating workloads for various hardware accelerators, including GPUs. Escobar et al. [12] outlined several requirements for handling large-scale IoT scenarios such as emergency rescue and mobile health monitoring and proposed a cloud–edge continuum concept that can be achieved by serverless computing. Chard et al. [13] introduced funcX, a federated function-as-a-service (FaaS) platform designed to support high-performance remote function execution across heterogeneous and distributed research infrastructures. These works present various design architectures for serverless edge computing, providing valuable insights. However, they do not introduce shared container instances in edge environments, potentially leading to heightened latency issues attributable to cold starts.

Building upon these architectural foundations, numerous studies have focused to enhance the performance of serverless edge computing systems. These efforts primarily aim at optimizing resource utilization, reducing latency, and improving the overall efficiency of serverless operations in edge environments. Rausch et al. [14] developed a container instances scheduling algorithm aimed at minimizing resource utilization and task completion time. Wu et al. [15] tried to reduce latency caused by cold start with a container lifecycle-aware scheduling strategy and implemented it on top of OpenWhisk, which is a serverless platform by Apache. Pan et al. [16] focused on the startup latency of containers. They proposed a retention-aware container caching in serverless edge computing by analyzing request distributions. Alkhailaileh et al. [17] constructed an optimization problem that can reduce the execution cost of data-intensive applications and energy consumption, taking into account the monetary cost and energy consumed by devices. Mampage et al. [18] proposed a deep reinforcement learning-based scheduling framework that optimally selects between serverless and serverful execution environments, leveraging an actor-critic architecture to minimize latency and user costs in hybrid cloud computing. Tang et al. [19] proposed a distributed task scheduling approach for serverless edge computing in IoT networks, formulating the problem as a partially observable stochastic game (POSG) and employing a multi-agent dueling double deep recurrent Q-network (D3RQN) to optimize task scheduling and resource allocation under limited observability. Shahrad et al. [20] analyzed real-world serverless workloads from Microsoft Azure, identifying key characteristics such as function invocation patterns, resource utilization, and cold-start behaviors, and proposed an optimized resource management policy to reduce cold starts while minimizing resource overhead. Fuerst et al. [1] introduced

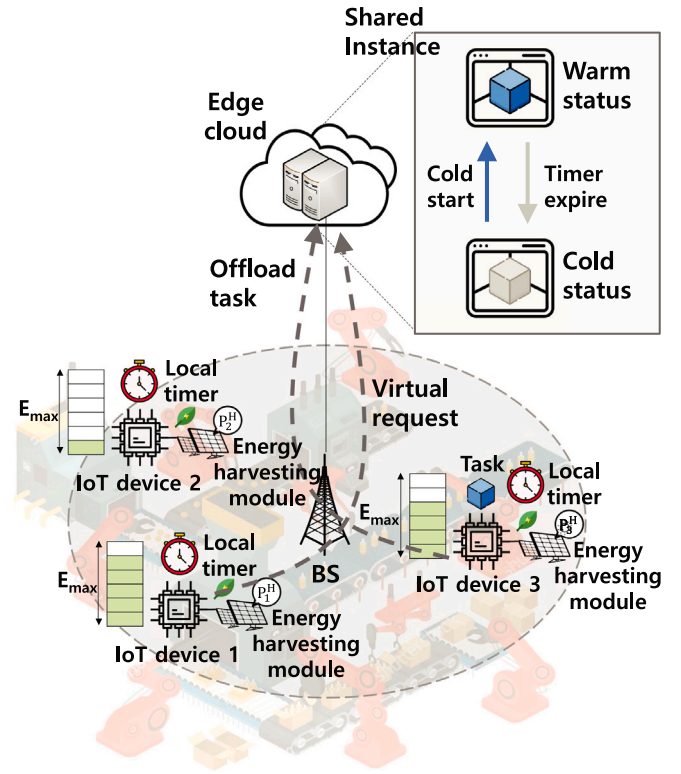


Fig. 1. System model.

FaaSCache, a caching-based approach to reducing cold-start latency in serverless computing, leveraging a greedy-dual caching framework to optimize keep-alive policies, significantly improving function latency and resource efficiency. Despite these optimizations, there remains a notable gap in understanding how to maintain container instances in a warm status through distributed approaches.

In addition to optimization techniques, considerable work has been focused on the performance analysis of serverless computing in edge environments. Palade et al. [21] conducted a comparative analysis of four commercial serverless frameworks, evaluating their performance, such as on-time task completion probability and throughput. Javed et al. [22] set up serverless platforms on a cluster of Raspberry Pis and evaluated their performance by simulating different types of edge workloads. Lloyd et al. [23] investigated the performance and cost associated with memory reservation sizes. They also assessed the scaling performance concerning growing concurrent workloads.

3. NA-SCIW framework

Fig. 1 illustrates the system model in which a system operator, such as a smart factory operator [24], deploys several IoT devices. These IoT devices are equipped with batteries that have a maximum capacity $E_{\max}$ and possess the ability to harvest energy from external sources, such as solar and wind power [25].³

In the NA-SCIW framework, IoT devices generate various types of tasks at regular intervals. These tasks are then processed using serverless computing with the concept of the shared container instance. That is, tasks of the same type are processed, even if they originate from different N_I IoT devices. When an IoT device generates a task,

³ It is worth noting that numerous modules for energy harvesting have been invented, making it convenient for IoT devices to incorporate energy harvesting capabilities [26].

Table 2
Summary of notations.

Notation	Description
N_I	Number of IoT devices generating the same type of the tasks
S	Overall state space
S_i	Local state space of IoT device i
E_i	State space for the energy status of IoT device i
C_i	State space for the last request timer of IoT device i
T_i	State space for the task status of IoT device i
H_i	State space for the energy harvesting condition of device i
$E_{\max}$	Battery capacity
$C_{\max}$	Cold start time
A	Overall action space
A_i	Local action space of IoT device i
p_H	Maximum energy harvesting probability
e_T	Energy consumption for transmitting the virtual request message
λ_T	Task generation rate
θ_T	Additional traffic overhead threshold
θ_E	Energy outage threshold

it sends an offloading request message, including its input data, to the edge cloud. Upon receiving the offloading request message, edge cloud checks whether the corresponding container instance for the task is in a warm status. If the container instance is not in a warm status, the edge cloud initiates a cold start to initialize the container instance, performs the task, and obtains the result. In the case where the corresponding container instance is already in a warm status, the edge cloud can directly execute the task and obtain the result without a cold start. Subsequently, the edge cloud returns the result to the IoT device.

As shown in this procedure, maintaining the corresponding container instance in a warm status contributes to reducing the overall task processing latency because the edge cloud can omit the cold start. However, if the container instance remains unused for a specific duration $C_{\max}$ (i.e., cold start time), the serverless computing operators transition it from a warm status to a cold status. Since the NA-SCIW framework introduces the shared container instance, even if only one request occurs from multiple IoT devices sharing the container instance, that instance can be maintained as a warm status (i.e., IoT devices do not experience the cold start). Nevertheless, if all IoT devices remain inactive for a certain period, the shared instance reverts to a cold status. To prevent this situation, the NA-SCIW framework introduces a cooperative keep warming mechanism. In this mechanism, each IoT device determines whether to send a virtual request message based on its own last request timer and the choices of nearby IoT devices. The last request timer tracks the time elapsed since the device's previous request transmission.

To factor in the decisions of neighboring devices, IoT devices use LoRa technology to share their current warming policies with each other [27]. LoRa technology is adopted for the policy sharing due to its long-range capability, always-on receive windows, and low power consumption, compared to alternative technologies such as BLE and Zigbee. These features make it well suited for distributed IoT environments where devices are spatially dispersed and require asynchronous reception of lightweight policy messages without tight coordination. In particular, LoRa Class C devices can keep their receive windows open until they send their next packet [27], allowing them to maintain continuous reception even after broadcasting their warming policies. Since the exchanged warming policies are lightweight approximately 8 bytes in size [28,29], this communication remains highly efficient and introduces negligible delay or congestion, which is essential for maintaining cooperative behavior in the NA-SCIW framework.

After receiving the warming policies from neighbors, each device can then estimate the cold start probability.⁴ By taking the cold start

⁴ A detailed explanation of how to derive the cold start probability is provided in Section 4.6.

probability into account, each IoT device adjusts its warming policy accordingly. Note that, in practice, the proposed distributed warming policy approach can be readily integrated into commercial edge platforms such as AWS Greengrass [30], Azure IoT Edge [31], and lightweight Kubernetes-based frameworks such as MicroK8s [32] and K3s [33]. Details are provided in Appendix.

4. Constrained stochastic game model

We formulated a constrained stochastic game model [34,35] to manage the warm status of the shared container instance in a distributed way.⁵ Within the context of the game, there are N_I participants, i.e., IoT devices. Table 2 provides a summary of the key notations.

4.1. State space

The overall state space S includes the individual state spaces of all players and can be described as the Cartesian product of these local state spaces, denoted $\prod_i S_i$. Additionally, S_i represents a finite local state space for IoT device i , which is defined by

$$S_i = E_i \times C_i \times T_i \times H_i \quad (1)$$

where E_i , C_i , and T_i denote the state spaces for the energy status, last request timer, and task status of IoT device i , respectively. In addition, H_i represents the state space for the energy harvesting condition of IoT device i . E_i is represented as

$$E_i = \{0, 1, \dots, E_{\max}\}. \quad (2)$$

C_i can be described as

$$C_i = \{0, 1, \dots, C_{\max}\}. \quad (3)$$

T_i can be denoted by

$$T_i = \{0, 1, 2\}. \quad (4)$$

Here, $T_i = 1$ (or $T_i = 0$) signifies whether IoT device i is generating (or not generating) a task. Meanwhile, $T_i = 2$ denotes that the task generated by IoT device i cannot be offloaded because the device has run out of energy.

H_i can be described as

$$H_i = \{0, 1, \dots, H_{\max}\}. \quad (5)$$

where $H_{\max}$ denotes the best possible energy harvesting condition.

4.2. Action space

The global action space A is defined as the Cartesian product of the individual local action spaces, expressed as $\prod_i A_i$, where A_i represents the finite local action space for IoT device i . Specifically, A_i can be described by

$$A_i = \{0, 1\} \quad (6)$$

where A_i indicates whether IoT device i transmits the virtual request message or not. That is, if $A_i = 1$, IoT device i transmits the virtual request message. Otherwise, the device does not transmit the message.

⁵ While advanced optimization techniques such as differential geometric optimization [36], fuzzy logic-based reliability allocation [37], and algebraic modeling based on minimal prime submodules [38] can be employed to manage the warm status of shared container instances, these approaches typically rely on system-wide coordination, require centralized computation, or are difficult to adapt to the dynamic and decentralized nature of IoT environments. In contrast, this paper adopts a constrained stochastic game formulation, which enables each IoT device to learn and update its warming policy independently based on local observations, while still ensuring global performance objectives through distributed optimization.

$$P[S'_i|S_i, A_i] = P[E'_i|E_i, T_i, H_i, A_i] \times P[C'_i|C_i, E_i, T_i, A_i] \times P[T'_i|T_i, E_i] \times P[H'_i|H_i]. \quad (7)$$

$$P[E'_i|E_i \neq E_{\max}, T_i = 0, H_i, A_i = 0] = \begin{cases} \frac{H_i}{H_{\max}} \times p_H, & \text{if } E'_i = E_i + 1 \\ 1 - (\frac{H_i}{H_{\max}} \times p_H), & \text{if } E'_i = E_i \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

$$P[E'_i|E_i = E_{\max}, T_i = 0, H_i, A_i = 0] = \begin{cases} 1, & \text{if } E'_i = E_i \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

$$P[E'_i|E_i \geq e_T, T_i = 0, H_i, A_i = 1] = \begin{cases} \frac{H_i}{H_{\max}} \times p_H, & \text{if } E'_i = E_i + 1 - e_T \\ 1 - (\frac{H_i}{H_{\max}} \times p_H), & \text{if } E'_i = E_i - e_T \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

$$P[E'_i|E_i \geq e_T, T_i \neq 0, H_i, A_i = 1] = \begin{cases} \frac{H_i}{H_{\max}} \times p_H, & \text{if } E'_i = E_i + 1 - e_T \\ 1 - (\frac{H_i}{H_{\max}} \times p_H), & \text{if } E'_i = E_i - e_T \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

Box I.

4.3. Transition probability

When IoT device i chooses to transmit the virtual request message, it consumes energy. Additionally, if IoT device i generates a task and has enough energy, it sends an offloading request message to the edge cloud. Consequently, the transition of E_i depends on the task state T_i , the current condition of energy harvesting H_i and the action A_i taken. At the same time, the last request timer is reset whenever IoT device i sends either the offloading request message or the virtual request message. Therefore, the transition of C_i is influenced by the energy state E_i , the task state T_i , and the action A_i . The offloading request message can only be transmitted if IoT device i has sufficient energy, meaning that the transition of T_i is affected by E_i . In summary, the transition probability $P[S'_i|S_i, A_i]$ from the current state $S_i = [E_i, C_i, T_i, H_i]$ to the next state $S'_i = [E'_i, C'_i, T'_i, H'_i]$, given that IoT device i performs action A_i , can be described by (7) given as in Box I.

If IoT device i neither generates a task nor transmits the virtual request message (i.e., $T_i = 0$ and $A_i = 0$), it has a chance to harvest one unit of energy with a probability $\frac{H_i}{H_{\max}} \times p_H$, where p_H represents the maximum energy harvesting probability. However, if the device's battery is already at full capacity, it cannot gain additional energy through harvesting. Therefore, the related transition probabilities are detailed in (8) and (9).

Meanwhile, when IoT device i transmits the virtual request message (i.e., $A_i = 1$) or has the task (i.e., $T_i \neq 1$), it consumes e_T . Consequently, the associated transition probabilities can be represented as (10) and (11) are given as in Box I. It is important to emphasize that IoT device i can only transmit the (virtual) request message if it has sufficient energy (i.e., $E_i \geq e_T$).

Despite IoT device i having the task (i.e., $T_i \neq 0$), it is unable to transmit the (virtual) request message if it lacks energy (i.e., $E_i < e_T$). In such a scenario, the battery level increases by one with a probability $\frac{H_i}{H_{\max}} \times p_H$. Consequently, $P[E'_i|E_i < e_T, T_i \neq 0, H_i, A_i]$ can be specified as

$$P[E'_i|E_i < e_T, T_i \neq 0, H_i, A_i] = \begin{cases} \frac{H_i}{H_{\max}} \times p_H, & \text{if } E'_i = E_i + 1 \\ 1 - (\frac{H_i}{H_{\max}} \times p_H), & \text{if } E'_i = E_i \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

When IoT device i does not transmit any request message, the last request timer increases by one until it reaches the maximum value (i.e., $C_{\max}$). In addition, if IoT device i lacks energy, preventing it from transmitting the request message, the last request timer also increases

by one. Consequently, the transition probabilities associated with these scenarios can be expressed as

$$P[C'_i|C_i, E_i, T_i = 0, A_i = 0] = \begin{cases} 1, & \text{if } C'_i = C_i + 1 \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

$$P[C'_i|C_i = C_{\max}, E_i, T_i = 0, A_i = 0] = \begin{cases} 1, & \text{if } C'_i = C_i \\ 0, & \text{otherwise} \end{cases} \quad (14)$$

$$P[C'_i|C_i, E_i < e_T, T_i, A_i] = \begin{cases} 1, & \text{if } C'_i = C_i + 1 \\ 0, & \text{otherwise} \end{cases} \quad (15)$$

and

$$P[C'_i|C_i = C_{\max}, E_i = 0, T_i, A_i] = \begin{cases} 1, & \text{if } C'_i = C_i + 1 \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

Meanwhile, when IoT device i has the necessary energy and transmits the (virtual) request message, the last request timer resets to 0. Thus, the corresponding transition probabilities can be formulated as

$$P[C'_i|C_i, E_i \geq e_T, T_i \neq 0, A_i] = \begin{cases} 1, & \text{if } C'_i = 0 \\ 0, & \text{otherwise} \end{cases} \quad (17)$$

and

$$P[C'_i|C_i, E_i \geq e_T, T_i, A_i = 1] = \begin{cases} 1, & \text{if } C'_i = 0 \\ 0, & \text{otherwise.} \end{cases} \quad (18)$$

The interval between successive task generations is assumed to follow an exponential distribution with a mean of $1/\lambda_T$ [39]. Consequently, the probability of generating a task within the decision epoch of duration τ is given by $\lambda_T \tau$ [40]. Thus, the corresponding transition probability can be delineated as

$$P[T'_i|T_i = 0, E_i] = \begin{cases} \lambda_T \tau, & \text{if } T'_i = 1 \\ 1 - \lambda_T \tau, & \text{if } T'_i = 0 \\ 0, & \text{otherwise.} \end{cases} \quad (19)$$

When IoT device i possesses the task and has available energy (i.e., $T_i = 1$ and $E_i \geq e_T$), it is capable of transmitting the request message. Subsequently, the task status transitions to 0. The associated transition probability can be expressed as

$$P[T'_i|T_i = 1, E_i \geq e_T] = \begin{cases} 1, & \text{if } T'_i = 0 \\ 0, & \text{otherwise.} \end{cases} \quad (20)$$

In the absence of energy (i.e., $E_i < e_T$), IoT device i is unable to transmit the request message, even if it possesses the task (i.e., $T_i = 1$).

Consequently, the task status transitions to 2. The relevant transition probability can be formulated as

$$P [T'_i | T_i = 1, E_i < e_T] = \begin{cases} 1, & \text{if } T'_i = 2 \\ 0, & \text{otherwise.} \end{cases} \quad (21)$$

When the task status is 2, the subsequent task state is determined based on the energy status. Specifically, if IoT device i has sufficient energy ($\geq e_T$), enabling it to transmit the request message, the task status transitions to 0. Otherwise, the task status persists as 2. The associated transition probabilities can be defined as

$$P [T'_i | T_i = 2, E_i \geq e_T] = \begin{cases} 1, & \text{if } T'_i = 0 \\ 0, & \text{otherwise} \end{cases} \quad (22)$$

and

$$P [T'_i | T_i = 2, E_i < e_T] = \begin{cases} 1, & \text{if } T'_i = 2 \\ 0, & \text{otherwise.} \end{cases} \quad (23)$$

In addition, since the energy harvesting condition H_i follows environmental dynamics (e.g., solar or vibrational energy availability), its transition probability can be defined in a statistical manner based on empirical data or predefined stochastic models.

4.4. Cost function

The cost function $r(S_i, A_i)$ can be formulated using the cold start probability. The situation where the last request timers of all IoT devices reach the cold start time indicates that the shared container instance has not been used during the cold start duration. Therefore, the cost function $r(S_i, A_i)$ can be expressed as

$$r(S_i, A_i) = \prod_i P [C_i = C_{\max}]. \quad (24)$$

The cold start probability refers to the likelihood that an IoT device fails to send a warming request before its last request timer reaches the maximum threshold $C_{\max}$. This probability can be estimated by observing how frequently the device enters such a state over time when following its current decision-making policy. Specifically, it is calculated based on the long-term behavior of the device, considering how often it chooses not to send a warming request while its timer continues to increase up to $C_{\max}$.

4.5. Constraint functions

The constraint functions are designed to ensure that the average extra traffic caused by virtual request messages and the energy outage probability of IoT device i remain below specified thresholds. Given that $A_i = 1$ signifies that IoT device i is transmitting a virtual request message, the first constraint function to control the average traffic overhead due to these messages, $c_T(S_i, A_i)$, can be defined as

$$c_T(S_i, A_i) = \begin{cases} 1, & \text{if } A_i = 1 \\ 0, & \text{otherwise.} \end{cases} \quad (25)$$

Meanwhile, $E_i = 0$ indicates that IoT device i is experiencing an energy outage. Hence, the constraint function for the energy outage probability of IoT device i , $c_E(S_i, A_i)$, can be formulated as

$$c_E(S_i, A_i) = \begin{cases} 1, & \text{if } E_i = 0 \\ 0, & \text{otherwise.} \end{cases} \quad (26)$$

4.6. Optimization formulation

Let π represent a stationary multi-policy for all IoT devices. The long-term average cold start probability can be defined as

$$\zeta_C(\pi) = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T E_{\pi} [r(S^t, A^t)] \quad (27)$$

where S^t and A^t denote the global state and the action taken at time t , respectively.

Additionally, IoT device i seeks to ensure that the average additional traffic overhead from virtual request messages stays below a certain threshold, while also keeping the average energy outage probability under a specified limit. Therefore, the long-term average additional traffic overhead and average energy outage probability can be expressed as

$$\xi_T(\pi) = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T E_{\pi} [c_T(S^t, A^t)] \leq \theta_T \quad (28)$$

and

$$\xi_E(\pi) = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T E_{\pi} [c_E(S^t, A^t)] \leq \theta_E \quad (29)$$

where θ_T and θ_E represent the thresholds for additional traffic overhead and energy outage probability, respectively.

Let π_i^* denote the optimal policy for IoT device i , given the policies of the other IoT devices. Specifically, for any stationary policies of the other IoT devices represented by π_{-i} , the best response policy π_i^* for IoT device i is defined such that $\zeta_C((\pi_i^*, \pi_{-i})) \leq \zeta_C((\pi_i, \pi_{-i}))$. If the condition $\zeta_C((\pi_i^*, \pi_{-i})) \leq \zeta_C((\pi_i, \pi_{-i}))$ holds for every IoT device for any feasible policy π_i such that (π_i, π_{-i}^*) meets all constraints, then the multi-policy $\pi^* = (\pi_i^*, \pi_{-i}^*)$ represents a constrained Nash equilibrium.

Consider a scenario where IoT devices repeatedly adjust their policies based on a best response strategy. In such a situation, the convergence of all IoT device policies implies a Nash equilibrium, denoted as $\pi^* = (\pi_i^*, \pi_{-i}^*)$, because $\zeta_C((\pi_i^*, \pi_{-i}^*)) \leq \zeta_C((\pi_i, \pi_{-i}^*))$ holds for all IoT devices, and the policy π_i^* for each device is converged. In this constrained Nash equilibrium, IoT device i cannot lower its cost (i.e., reduce the cold start probability) by switching to a different stationary policy π_i without either breaching the constraints or causing the other IoT devices to change their stationary policies. Note that, in a stochastic game with a finite number of players, finite action sets, and a finite set of states, there is always a Nash equilibrium.

Let $\phi_{i, \pi_{-i}}(S_i, A_i)$ represent the stationary probability of IoT device i choosing action A_i in local state S_i , given that other IoT devices follow stationary policies π_{-i} . A similar linear programming (LP) model can then be constructed, with its solution $\phi_{i, \pi_{-i}}^*(S_i, A_i)$ interpreted as the optimal policy for the stochastic game. The LP model is formulated as follows:

To minimize the cold start probability, the objective function can be formulated as

$$\min_{\phi(S, A)} \sum_S \sum_A \phi_{i, \pi_{-i}}(S_i, A_i) r(S_i, A_i). \quad (30)$$

Due to the necessity of keeping the additional traffic overhead caused by virtual request messages below the specified threshold, we have

$$\sum_S \sum_A \phi_{i, \pi_{-i}}(S_i, A_i) c_T(S_i, A_i) \leq \theta_T. \quad (31)$$

To ensure that the average energy outage probability remains below the designated threshold θ_E , the corresponding constraint can be derived as

$$\sum_S \sum_A \phi_{i, \pi_{-i}}(S_i, A_i) c_E(S_i, A_i) \leq \theta_E. \quad (32)$$

The constraint regarding the Chapman–Kolmogorov equation is described as

$$\sum_A \phi_{i, \pi_{-i}}(S'_i, A_i) = \sum_S \sum_A \phi_{i, \pi_{-i}}(S_i, A_i) P[S'_i | S_i, A_i]. \quad (33)$$

For the basic principles of probability, we have

$$\sum_S \sum_A \phi_{i, \pi_{-i}}(S'_i, A_i) = 1 \quad (34)$$

and

$$\phi_{i, \pi_{-i}}(S'_i, A_i) \geq 0. \quad (35)$$

With the optimal solution of the above LP problem $\phi_{i,\pi_i}^*(S_i, A_i)$, the best response policy for IoT device i is derived as

$$\pi_i^*(S_i, A_i) = \frac{\phi_{i,\pi_i}^*(S_i, A_i)}{\sum_{A'_i \in \mathcal{A}_i} \phi_{i,\pi_i}^*(S_i, A'_i)}. \quad (36)$$

While the current formulation assumes homogeneous task types for analytical tractability, the NA-SCIW framework can be extended to support heterogeneous tasks with varying resource requirements or priorities. This can be achieved by generalizing the state and action spaces to include task-type identifiers and by refining the cost function to reflect task-specific constraints. For instance, in heterogeneous environments, tasks can be weighted differently based on their urgency or computational load, enabling differentiated warming policies. This extensibility makes NA-SCIW adaptable to a broader range of real-world IoT deployments, such as multimodal smart city infrastructures or complex healthcare monitoring systems.

To illustrate the extensibility of NA-SCIW to heterogeneous task environments, we discuss a potential augmentation of the device state space to incorporate task type information:

$$S_i = \mathbf{E}_i \times \mathbf{C}_i \times \mathbf{T}_i \times \mathbf{H}_i \times \tau_i \quad (37)$$

where $\tau_i \in \{1, 2, 3, 4\}$ is the task type index indicating the category of the assigned task. Specifically, $\tau_i = 1$ represents a CPU-intensive task, $\tau_i = 2$ a memory-intensive task, $\tau_i = 3$ an I/O-intensive task, and $\tau_i = 4$ a network-intensive task. This augmented state representation enables the warming policy to adapt based on task-specific priorities.

Building upon this extensibility, future work may explore task-aware policy adaptation mechanisms. One approach is to assign differentiated weights to the cost or reward functions based on task priorities, such as real-time urgency or energy sensitivity. In reinforcement learning-based implementations, Q-value updates could be adjusted to emphasize higher-priority tasks, enabling IoT devices to learn warming policies that align with critical service requirements. These mechanisms would allow NA-SCIW to flexibly support diverse service-level objectives and enable priority-aware resource management in heterogeneous IoT deployments.

Algorithm 1 Best Response Dynamics-Based Policy Optimization Algorithm

- 1: **Initialization:**
 - 2: Load system parameters $(p_H, \lambda_T, \theta_T, \theta_E)$ and failure probabilities p_{fail} for each device.
 - 3: Generate a random initial policy $\pi_i^{(0)}$ for each IoT device i .
 - 4: Set iteration counter $t \leftarrow 0$.
 - 5: **repeat**
 - 6: $t \leftarrow t + 1$
 - 7: **for** each IoT device i **do parallel**
 - 8: Transmit current policy $\pi_i^{(t-1)}$ to neighboring IoT devices.
 - 9: Calculate the cold start probability $r_i(S, A)$.
 - 10: solve the LP problem to obtain best response policy $\pi_i^{(t)}$.
 - 11: **end for**
 - 12: **until** All policies of IoT devices are converged
-

To determine the optimal policies for IoT devices, we propose an algorithm based on best response dynamics (refer to Algorithm 1). Initially, each IoT device randomly selects its policy and loads the system parameters $(p_H, \lambda_T, \theta_T, \theta_E)$ along with the policy-sharing failure probability p_{fail} (lines 1–3 in Algorithm 1). The iteration counter t is also initialized (line 4 in Algorithm 1). Subsequently, each device shares its current policy with neighboring devices (line 8 in Algorithm 1). Based on the received policies,⁶ each device computes the cold start

probability as $r(S, A) = \prod_i P[C_i = C_{\max}]$. This is further evaluated using the solution from the LP model as $\prod_i \sum_{S_i: C_i = C_{\max}} \sum_{A_i} \phi_{i,\pi_i}^*(S_i, A_i)$ (line 9 in Algorithm 1). Each IoT device then solves a local LP problem to derive the best response policy π_i^* (line 10 in Algorithm 1). This process is repeated until the policies of all IoT devices converge. Importantly, such convergence is theoretically guaranteed by the structure of the proposed NA-SCIW framework. The underlying model is formulated as a constrained stochastic game with a finite number of players, states, and actions per IoT device. It is well known that in such finite-state, finite-action games, best response dynamics converge to a pure-strategy Nash equilibrium within a finite number of iterations [41]. This theoretical property is also empirically validated in our simulation study (see Section 5.1).

The computational complexity for solving the LP problem is polynomial with Vaidya's algorithm [42], specifically $\mathcal{O}(\Omega(S_i) \cdot \Omega(A_i)^3)$, where $\mathcal{O}(\cdot)$ denotes Big O notation and $\Omega(X)$ represents the number of elements in the set X . Importantly, the LP problem is independently solved by an IoT device using only local state and action spaces. Therefore, the per-device computational overhead remains low, regardless of the overall network size. Additionally, as detailed in Section 5.1, stationary policies can be reached within a few iterations, further supporting real-time applicability.

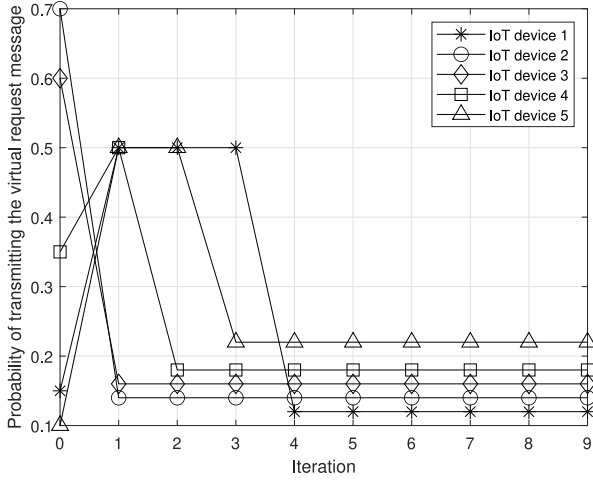
To validate the practical feasibility of NA-SCIW, we also measured the runtime overhead of solving the local LP problems in our simulation environment. The simulation was conducted on a desktop equipped with a 12th Gen Intel® Core™ i7-12700K processor and 64 GB RAM. Under this setup, the average time to solve each LP problem was approximately 0.8778 s, confirming that the proposed method can be applied in near real-time scenarios. While alternative lightweight methods such as Q-learning may offer lower per-decision latency, they often require iterative training and do not provide strict guarantees for constraint satisfaction. In contrast, our LP-based solution achieves rapid convergence with guaranteed constraint adherence, making it a practical and reliable choice for resource constrained IoT environments.

5. Evaluation results

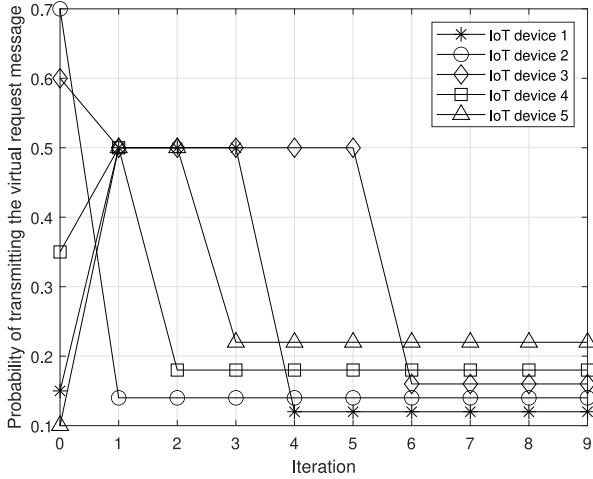
To evaluate performance, we compare NA-SCIW with four alternative schemes: (1) ALWAYS, where IoT devices transmit the virtual request message whenever they have any energy; (2) NONE, where IoT devices never transmit the virtual request message regardless of their energy status; (3) RAND, where IoT devices transmit the virtual request message with a random probability; (4) PROB, where IoT devices transmit the virtual request message with a predefined probability; and (5) MARL, where each IoT device independently learns and follows its own policy using a deep Q-network (DQN) in a multi-agent reinforcement learning environment [43]. Note that, since no prior work addresses the shared container instance, we created these simple comparison schemes to assess the performance of NA-SCIW.

The default parameter settings are as follows. Due to the lack of references for some parameters, we performed extensive evaluations using a range of settings. The number of IoT devices, N_I , is set to 5. The energy required to transmit the virtual request message, e_T , is normalized to 1. The maximum battery capacity, $E_{\max}$, is assumed to be 20. The cold start timer, $C_{\max}$, is set to 5. The maximum energy harvesting probability p_H is set to 0.8, and the energy harvesting condition H_i is discretized into three levels and is updated probabilistically at each decision epoch to reflect the time-varying nature of energy harvesting conditions. The harvesting level is sampled from a predefined distribution that captures typical variations over time (e.g., diurnal patterns) [44–47]. Meanwhile, based on the Azure functions invocation trace 2021 dataset [48,49], the task generation rates are empirically set in the range of 0.09 and 0.44. Both the additional traffic overhead and energy outage probability thresholds are set to 0.1.

⁶ In the event of a policy-sharing failure, the fallback value from the previous round is used.



(a) Without a policy sharing failure probability.



(b) With a policy sharing failure probability.

Fig. 2. Converging process to Nash equilibrium.

5.1. Convergence to Nash equilibrium

Fig. 2(a) demonstrates the convergence to Nash equilibrium under ideal conditions without policy-sharing failures within a epoch. In this result, the energy harvesting probabilities for IoT devices 1 through 5 are set to 0.05, 0.1, 0.15, 0.2, and 0.25, respectively. As shown in the figure, it takes only four iterations for the policies of all IoT devices to converge to the Nash equilibrium. This result suggests that the proposed algorithm based on best response dynamics can be practically applied with minimal signaling overhead.

The converged policy (i.e., the probability of transmitting the virtual request message) for each IoT device is determined by considering both its own energy harvesting probability and the current policies of the other devices. Specifically, since IoT device 1 has the smallest energy harvesting probability (i.e., 0.05), and the other devices exhibit relatively high transmission probabilities, its optimal strategy is to transmit with the lowest probability. This reflects a rational equilibrium strategy that balances energy availability and system-wide cold start mitigation.

In real-world deployments, LoRa communication delays or packet losses may hinder timely policy synchronization within a time slot, potentially affecting convergence and overall system performance. To

Table 3

Number of iterations to converge N_C under different number of IoT device N_I .

N_I	10	30	50	70	90
N_C	9	29	49	69	88

capture this effect, we model the impact of such communication issues using a policy-sharing failure probability, which represents the likelihood that an IoT device fails to receive updated policies from its neighbors within a time slot due to delay or packet loss.⁷ The convergence behavior under this condition is illustrated in Fig. 2(b). In this experiment, the failure probability for each IoT device is set in the range between 0.1 and 0.4. As shown in Fig. 2(b), despite the presence of sharing failures, the proposed algorithm still converges well, demonstrating its robustness to communication delays or packet losses.

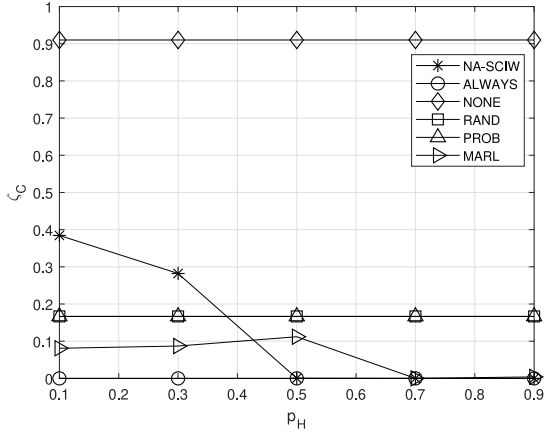
To further quantify the convergence behavior of Algorithm 1, we report the average number of iterations required to reach Nash equilibrium N_C under varying numbers of IoT devices N_I . Table 3 summarizes these results, demonstrating that the algorithm maintains reasonable number of iterations even as the network size increases.

5.2. Effect of p_H

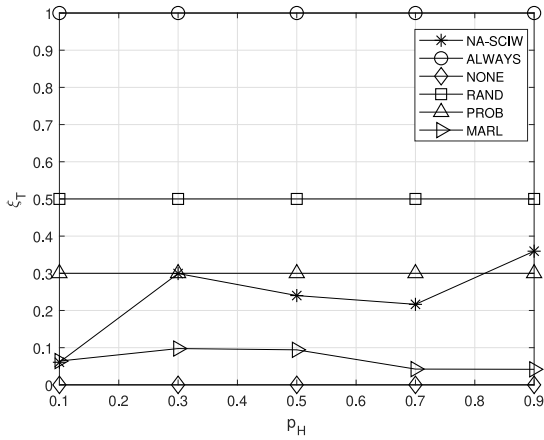
Fig. 3 shows the effect of the energy harvesting probability. In this result, the threshold for the additional traffic overhead is set to 1. From Fig. 3(a), it can be found that the average cold start probability of NA-SCIW decreases with the increase in the energy harvesting probability. This can be explained as follows: a higher energy harvesting probability indicates that IoT devices can transmit the virtual request message without concern about energy depletion. NA-SCIW recognizes this fact, and thus it adaptively adjusts the transmission policy on the virtual request message. Especially in this situation (i.e., with a higher energy harvesting probability), IoT devices tend to transmit virtual request messages more frequently. This phenomenon can be found in Fig. 3(b), which shows that the resulting traffic overhead caused by virtual request messages tends to increase as the energy harvesting probability rises. This implies that IoT devices are more inclined to utilize their harvested energy for transmitting such messages under favorable energy conditions. Even though the IoT device transmits more virtual request messages as the harvesting probability increases, the energy outage probability decreases due to sufficient energy harvesting, as shown in Fig. 3(c). It is worth noting that NA-SCIW keeps the energy outage probability below the target level of 0.1, even under challenging conditions with a low energy harvesting probability.

In contrast, the other comparison schemes except MARL (i.e., ALWAYS, NONE, RAND, and PROB) use fixed transmission policies that do not account for the energy harvesting probability. As a result, their average cold start probabilities, additional traffic overheads, and energy outage probabilities remain constant regardless of the energy harvesting conditions. Meanwhile, MARL does not effectively exploit the correlation between energy harvesting probabilities and transmission opportunities. As a result, it tends to learn conservative policies even under favorable energy conditions, leading to suboptimal performance in reducing cold starts.

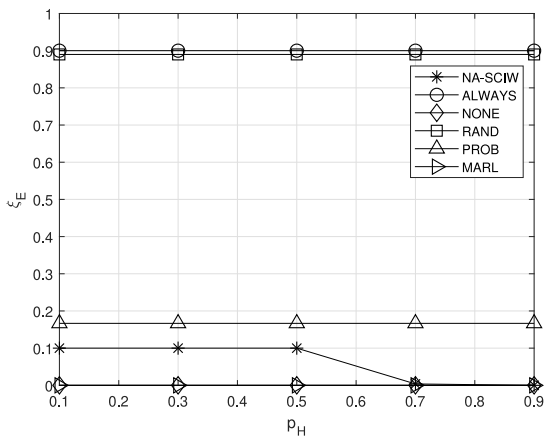
⁷ The introduced policy-sharing failure probability can also be interpreted as a general abstraction for untrustworthy behavior or disrupted cooperation (e.g., due to malicious or faulty devices). This provides a preliminary insight into the resilience of the proposed algorithm under such non-ideal conditions.



(a) Average cold start probability.



(b) Average additional traffic overhead.

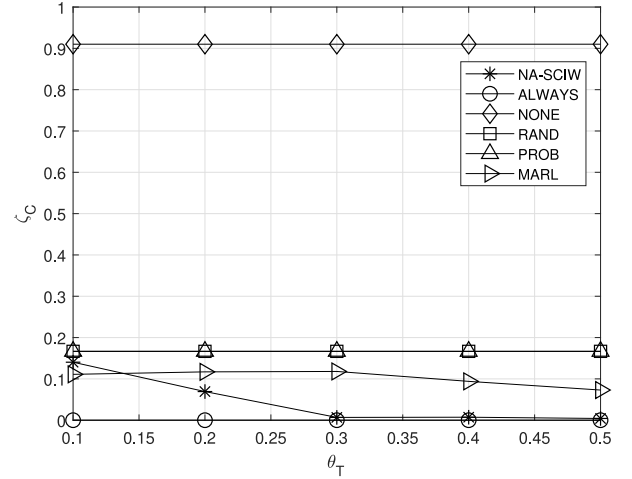


(c) Average energy outage probability.

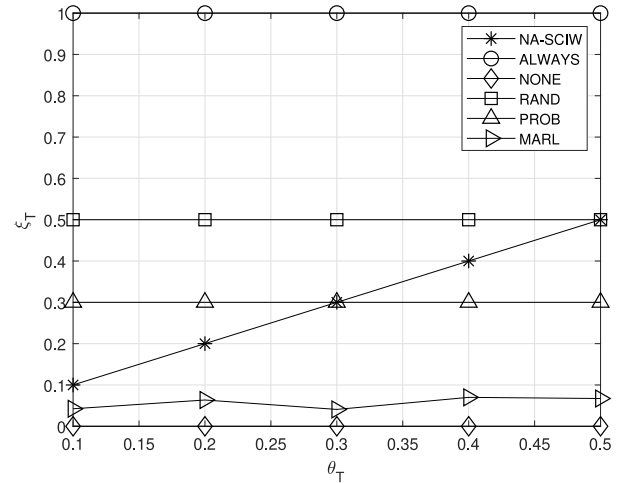
Fig. 3. Effect of the energy harvesting probability.

5.3. Effect of θ_T

Fig. 4 illustrates the impact of the additional traffic overhead threshold. As seen in Fig. 4(a) and (b), NA-SCIW effectively reduces the cold start probability while keeping the additional traffic overhead within the specified threshold, θ_T . This is because a higher θ_T allows IoT devices to transmit the virtual request message more often.



(a) Average cold start probability.



(b) Average additional traffic overhead.

Fig. 4. Effect of the additional traffic overhead threshold.

Consequently, NA-SCIW enables IoT devices to adjust their message transmission frequency based on this allowance. In contrast, MARL does not fully utilize the allowable overhead θ_T . Although DRL agents are trained under the updated threshold settings—which permit more frequent virtual request transmissions—they still fail to recognize the benefit of actively leveraging this flexibility to reduce cold starts. This is likely due to the conservative nature of the learned policy or the sparsity of reward signals associated with cold start events, which leads the agents to underutilize the available transmission opportunities. As a result, the cold start probability remains largely unchanged even when the system allows for higher traffic overhead.

5.4. Effect of λ_T

Fig. 5 illustrates how the task generation rate affects the average cold start probability. As depicted in the figure, the average cold start probability of NA-SCIW rises with an increasing task generation rate. This trend can be explained as follows: when the task generates, IoT devices always transmit the offloading request messages by consuming their energy. Thus, given a high task generation rate, it is not possible for IoT devices to maintain sufficient energy to transmit the virtual

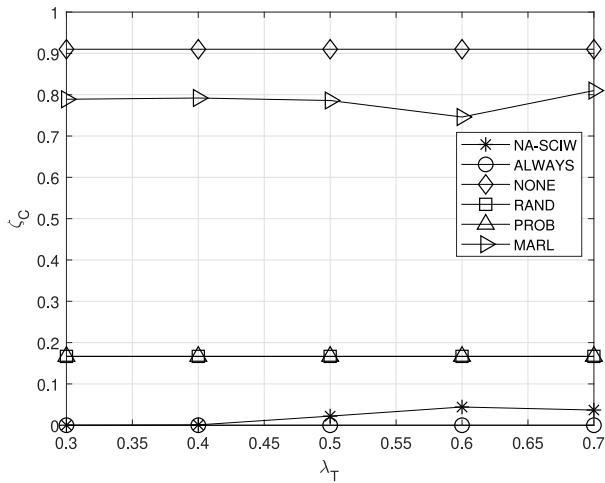


Fig. 5. Effect of the task generation rate on the average cold start probability.

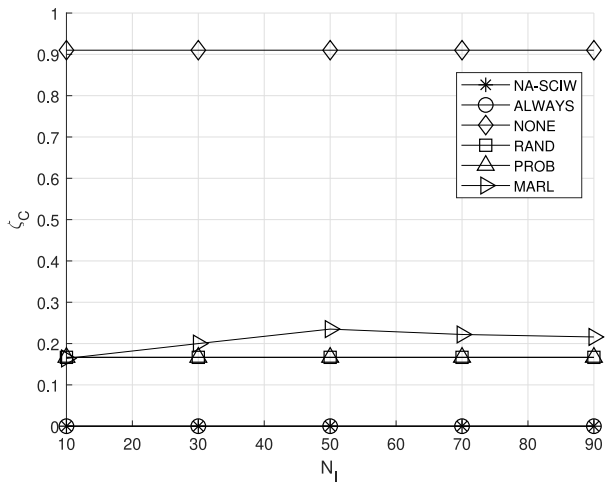


Fig. 6. Effect of the number of IoT devices on the average cold start probability.

request messages whenever necessary, leading to the increased cold start probability.

5.5. Effect of N_I

Fig. 6 shows the cold start probability as the number of IoT devices increases. As shown in the figure, NA-SCIW consistently maintains low cold start probability even as the number of devices grows, demonstrating its stable performance in larger deployments. This is because policy optimization in NA-SCIW is performed locally at each IoT device without centralized coordination, ensuring that the computational complexity per device remains unaffected by the total number of devices and that the proposed best response dynamics-based policy optimization algorithm converges well even as the number of devices increases.

In contrast, MARL shows an increase in cold start probability as the number of devices grows. This is because DRL agents are trained on a fixed neural network size and struggle to fully capture the increasingly complex interaction patterns that emerge among a larger number of devices. Meanwhile, the performance of other comparison schemes (i.e., ALWAYS, NONE, RAND, and PROB) remains largely unchanged with the number of devices, since their policies are fixed and do not adapt to the network scale.

6. Conclusions

In this paper, we introduce a neighbor-aware shared container instance warming framework (NA-SCIW). NA-SCIW utilizes two main concepts: (1) shared container instances, and (2) a collaborative keep-warming mechanism. By sharing a container instance among multiple IoT devices, even if only one request is generated, the instance can be maintained in a warm status, effectively preventing cold starts for IoT devices. However, if all IoT devices refrain from generating requests for a specific duration, the shared instance reverts to a cold status. To tackle this issue, the NA-SCIW framework employs a cooperative keep-warming mechanism, where each IoT device determines whether to send a virtual request message based on the choices made by neighboring devices. We develop a constrained stochastic game model aimed at minimizing the cold start probability while ensuring that the average additional traffic overhead from virtual request messages and the energy outage probability of IoT devices remain within predefined limits. To solve this model, we introduce an algorithm based on best response dynamics. The evaluation results show that NA-SCIW effectively reduces the cold start probability while keeping both the average additional traffic overhead and energy outage probability low for IoT devices.

As a future research direction, we plan to explore hybrid communication strategies, as the size or frequency of warming policy updates can grow in complex or large-scale deployments. In such cases, combining LoRa for transmitting lightweight metadata (e.g., policy indices) with WiFi or BLE for larger updates may enhance communication efficiency and reliability, particularly under dynamic policy adaptation scenarios. Furthermore, to enhance the resilience of NA-SCIW under adversarial conditions or faulty nodes, future work will also consider incorporating lightweight Byzantine fault tolerance (BFT) mechanisms, such as trust scoring or policy verification, to ensure robust cooperation even when policy messages are unreliable. Beyond this, emerging research trends offer valuable extensions to NA-SCIW. Green cloud multimedia networking can inform energy-aware warming policies, while hybrid scheduling methods such as the aquila optimizer (AO) and whale optimization algorithm (WOA) highlight trade-offs between centralized and distributed control. Multi-objective optimizers like the Harris hawks optimization (HHO) algorithm may support joint optimization of latency, energy, and cost. Lastly, integrating software-defined networking (SDN) could enable more flexible and context-aware policy enforcement. Together, these directions enhance NA-SCIW's potential for scalable and adaptive edge-cloud resource management.

CRedit authorship contribution statement

Yumi Kim: Writing – original draft, Software, Methodology.
Bokyeong Kim: Writing – original draft, Visualization, Validation.
Taewon Song: Methodology, Conceptualization. **Haneul Ko:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Haneul Ko reports financial support was provided by National Research Foundation of Korea. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Integration of NA-SCIW with real-world edge computing platforms

To demonstrate the practical applicability and scalability of the proposed NA-SCIW framework, we describe how it can be integrated into representative commercial edge computing platforms, including AWS Greengrass [30], Azure IoT Edge [31], and lightweight Kubernetes-based frameworks such as MicroK8s [32] and K3s [33].

(1) AWS Greengrass integration

NA-SCIW can be deployed as a Lambda Function within the AWS Greengrass Core runtime environment. The framework leverages the native Greengrass APIs to manage container instance states (e.g., Alive, Idle) based on actual or virtual request arrivals. Specifically, Greengrass provides Lambda lifecycle management tools that enable NA-SCIW to keep functions warm by pre-invoking them or adjusting timeout thresholds. The policy-sharing and energy-aware warming logic can be encapsulated as a part of the Lambda codebase, interacting with local resource monitoring agents.

(2) azure IoT edge integration

On Azure IoT Edge, NA-SCIW can be implemented as a custom IoT Edge Module. The Edge Hub serves as the broker for all inter-module and device-to-cloud communications. The warming strategy is coordinated through the Module Twin interface, which allows runtime configuration updates and state synchronization. Energy status, task generation, and warming decisions can be published via MQTT or the built-in messaging framework. This design supports modular deployment and remote management of warming policies.

(3) MicroK8s/K3s integration

For lightweight Kubernetes-based deployments, NA-SCIW can be containerized and executed as a Pod. The warming mechanism can be enforced at the Pod level using sidecar containers or init containers that maintain container state and implement energy-aware control logic. To enable scalable and declarative management, NA-SCIW can be extended via Kubernetes custom resource definitions (CRDs), allowing administrators to define container warming policies as first-class resources in the cluster. This approach enables compatibility with both centralized controllers and edge-side agents.

Data availability

The authors do not have permission to share data.

References

- [1] A. Fuerst, P. Sharma, FaasCache: Keeping serverless computing alive with greedy-dual caching, in: Proc. ACM ASPLOS 2021, 2021.
- [2] A. Kumari, et al., Mitigating cold-start delay using warm-start containers in serverless platform, in: Proc. IEEE INDICON 2022, 2022.
- [3] P. Sliva, et al., Prebaking functions to warm the serverless cold start, in: Proc. Middleware 2020, 2020.
- [4] H. Ko, LPGA: Location privacy-guaranteed offloading algorithm in cache-enabled edge clouds, IEEE TCC 10 (4) (2022) 2729–2738.
- [5] S. Nastic, et al., A serverless real-time data analytics platform for edge computing, IEEE Internet Comput. 21 (4) (2017) 64–71.
- [6] A. Pérez, et al., Serverless Computing for Container-Based Architectures, Vol. 83, Elsevier FGCS, 2018, pp. 50–59.
- [7] A. Das, et al., Performance optimization for edge-cloud serverless platforms via dynamic task placement, in: Proc. IEEE/ACM CCGRID 2020, 2020.
- [8] C. Cicconetti, et al., Toward distributed computing environments with serverless solutions in edge systems, IEEE Commun. Mag. 58 (3) (2020) 40–46.
- [9] M.S. Aslanpour, et al., Serverless Edge Computing: Vision and Challenges, 2021.
- [10] S.R. Poojara, et al., Serverless Data Pipeline Approaches for IoT Data in Fog and Cloud Computing, Vol. 130, Elsevier FGCS, 2022, pp. 91–105.
- [11] S. Werner, T. Schirmer, Hardless: A generalized serverless compute architecture for hardware processing accelerators, in: Proc. IEEE IC2E 2022, 2022.
- [12] J.J.L. Escobar, et al., Decentralized serverless IoT dataflow architecture for the cloud-to-edge continuum, in: Proc. ICIN 2023, 2023.
- [13] R. Chard, et al., Funcx: A federated function serving fabric for science, in: Proc. ACM HPDC, 2020.
- [14] T. Rausch, A. Rashed, S. Dustdar, Optimized Container Scheduling for Data-Intensive Serverless Edge Computing, Vol. 114, Elsevier FGCS, 2021, pp. 259–271.
- [15] S. Wu, et al., Container lifecycle-aware scheduling for serverless computing, Wiley Softw.: Pr. Exp. 52 (2) (2022) 337–352.
- [16] L. Pan, et al., Retention-aware container caching for serverless edge computing, in: Proc. IEEE INFOCOM 2022, 2022.
- [17] M. Alkhalaileh, et al., Data-Intensive Application Scheduling on Mobile Edge Cloud Computing, Vol. 167, Elsevier JNCA, 2020, 102735.
- [18] A. Mampage, et al., Deep reinforcement learning for scheduling applications in serverless and serverful hybrid computing environments, IEEE TSC 18 (2) (2025) 718–728.
- [19] Q. Tang, et al., Distributed task scheduling in serverless edge computing networks for the internet of things: A learning approach, IEEE IoT- J 9 (20) (2022) 19634–19648.
- [20] M. Shahrad, et al., Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider, in: Proc. USENIX ATC, 2020.
- [21] A. Palade, A. Kazmi, S. Clarke, An evaluation of open source serverless computing frameworks support at the edge, in: Proc. IEEE SERVICES 2019, 2019.
- [22] H. Javed, et al., Serverless platforms on the edge: A performance analysis, in: Springer New Frontiers in Cloud Computing and Internet of Things, 2022, pp. 165–184.
- [23] W. Lloyd, et al., Improving application migration to serverless computing platforms: Latency mitigation with keep-alive workloads, in: IEEE/ACM UCC Companion 2018, 2018.
- [24] C. Nagpal, et al., IIoT based smart factory 4.0 over the cloud, in: Proc. ICCICE 2019, 2019.
- [25] H. Heidari, et al., Energy harvesting and power management for IoT devices in the 5G era, IEEE Commun. Mag. 59 (9) (2021) 91–97.
- [26] J. Lee, H. Ko, Neighbor-aware distributed task offloading algorithm in energy harvesting internet of things, IEEE IoT- J 10 (10) (2023) 8744–8753.
- [27] LoRa Alliance, LoRaWAN L2 1.0.4 Specification, [Online] Available: https://loralliance.org/resource_hub/lorawan-104-specification-package/.
- [28] LoRa Alliance, LoRaWAN™ 1.1 specification, 2017, [Online]. Available: <https://loralliance.org>.
- [29] IEEE Computer Society, IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2008 (Revision of IEEE Std 754-1985), 2008.
- [30] AWS, What is AWS IoT Greengrass?, [Online] Available: <https://docs.aws.amazon.com/greengrass/v2/developerguide/what-is-iot-greengrass.html>.
- [31] Microsoft, Microsoft, What is Azure IoT Edge, [Online] Available: <https://learn.microsoft.com/en-us/azure/iot-edge/about-iot-edge>.
- [32] MicroK8s, MicroK8s documentation, [Online] Available: <https://microk8s.io/docs>.
- [33] K3S, What is K3s?, [Online] Available: <https://docs.k3s.io/>.
- [34] S. Chen, et al., Distributed computation offloading based on stochastic game in multi-server mobile edge computing networks, in: Proc. SmartIoT, 2019.
- [35] A. Seid, et al., Multi-agent DRL for task offloading and resource allocation in multi-UAV enabled IoT edge network, IEEE TNSM 18 (4) (2021) 4531–4547.
- [36] S.A. Abed, et al., Structural reliability and optimization using differential geometric approaches, Iraqi J. Comput. Sci. Math. 5 (1) (2024) 7.
- [37] S.A. Abed, Z.H. Khalil, Reliability allocation in complex systems using fuzzy logic modules, Babylon. J. Math. 2023 (2023) 78–83.
- [38] R. Al-Masroub, M.S. Fiadh, Minimal prime and semiprime submodules, Babylon. J. Math. 2023 (2023) 84–87.
- [39] M. Adhikari, M. Mukherjee, S. Srirama, DPOT: A deadline and priority-aware task offloading in fog computing framework leveraging multilevel feedback queueing, IEEE IoT- J 7 (7) (2020) 5773–5782.
- [40] H. Ko, M. Jo, V. Leung, Application-aware migration algorithm with prefetching in heterogeneous cloud environments, IEEE TCC 10 (4) (2022) 2324–2333.
- [41] D. Monderer, L.S. Shapley, Potential Games, vol. 14, Elsevier Games and Economic Behavior, 1996, pp. 124–143.
- [42] Vaidya's algorithm, 2022, (Accessed 17 December 2023). [Online] Available: https://algorithm-wiki.csail.mit.edu/wiki/Vaidya%27s_Algorithm.
- [43] T. Li, et al., Applications of multi-agent reinforcement learning in future internet: A comprehensive survey, IEEE Commun. Surv. & Tutorials 24 (2) (2022) 1240–1279, Secondquarter.
- [44] W. Zhu, et al., Transmission policies for energy harvesting sensors based on Markov chain energy supply, EAI Endorsed Trans. Energy Web 3 (8) (2015).

- [45] H. Kong, et al., On the performance of wireless energy harvesting networks in a boolean-Poisson model, in: Proc. IEEE ICC, 2016.
- [46] Z. Ling, et al., Age-of-information minimization in healthcare IoT using distributionally robust optimization, IEEE IoT- J 9 (17) (2022) 16154–16167.
- [47] M. Tavana, et al., Multi-site energy harvesting for battery-less IoT devices: Prospects and limits, in: Proc. VTC 2022-Fall, 2022.
- [48] Y. Zhang, et al., Faster and cheaper serverless computing on harvested resources, in: Proc. ACM SOSP, 2021.
- [49] AzureDataset2021, 2025, [Online]. Available: <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>.



Yumi Kim received the B.S. degree from the Department of Computer Convergence Software, Korea University, Sejong, Korea, in 2024. She is currently a M.S/Ph.D integrated course student at the Department of Electronics and Information Convergence Engineering, Kyung Hee University, Yongin, Korea. Her research interests include 5G/6G networks, Internet of Things (IoT), reinforcement learning, mobile cloud computing, split computing and SDN/NFV.



Bokyeong Kim received a B.S. degree from the Department of Computer Convergence Software, Korea University, Sejong, Korea, in 2022. She is currently enrolled in an M.S. course at the Department of Electronics and Information Convergence Engineering, Kyung Hee University, Yongin, Korea. Her research interests include 5G/6G networks, Internet of Things (IoT), reinforcement learning, mobile cloud computing, and split computing.



Taewon Song received the B.S. and Ph.D. degrees in electrical engineering from Korea University, Seoul, South Korea, in 2010 and 2017, respectively. In 2021, he joined Soonchunhyang University, where he is currently an assistant professor in the Department of IoT, College of SW Convergence. After receiving LG Electronics Scholarship, he was a senior researcher at the Advanced Standard R&D Laboratory, LG Electronics, where he worked on standardization for IEEE 802.11ba Wake-Up Radio and IEEE 802.11be Extremely High Throughput from 2017 to 2020. He was a visiting student supported by the BK21+ project with the University of Florida from February 2015 to April 2015. From January 2012 to February 2012, he was a visiting researcher supported by the National Research Foundation of Korea at the National Institute of Information and Communications Technology, Japan. His current research interests include AI-empowered networking, low-power wake-up radio, and the wireless medium access control protocol of next-generation WLANs.



Haneul Ko [SM'23] received B.S. and Ph.D. from the School of Electrical Engineering, Korea University, Seoul, Korea, in 2011 and 2016, respectively. He is currently an assistant professor in the Department of Electronic Engineering, Kyung Hee University, Yongin, Korea. From 2019 to 2022, he was an assistant professor in the Department of Computer and Information Science, Korea University, Sejong, Korea. From 2017 to 2018, he was a Postdoctoral Fellow at the University of British Columbia, Vancouver, BC, Canada. From 2016 to 2017, he was a Postdoctoral Fellow in mobile network and communications, Korea University, Seoul, Korea. He was the recipient of Minister of Education Award in 2019, IEEE ComSoc APB Outstanding Young Researcher Award in 2022, and Korean Institute of Communications and Information Sciences (KICS) Haedong Young Scholar Award 2023. He is a senior member of IEEE. He is an editor of IEEE Transactions on Machine Learning in Communications and Networking. His research interests include 5G/6G networks, network automation, mobile cloud computing, SDN/NFV, and Future Internet.